Mi primera prueba con PHPUnit

Una historia sencilla.
Una suma.
Después, nuestro CRUD.



Una tarea muy sencilla

Sumar 2 y 3

2 + 3 = 5

Sabemos qué debería dar antes de ejecutar.



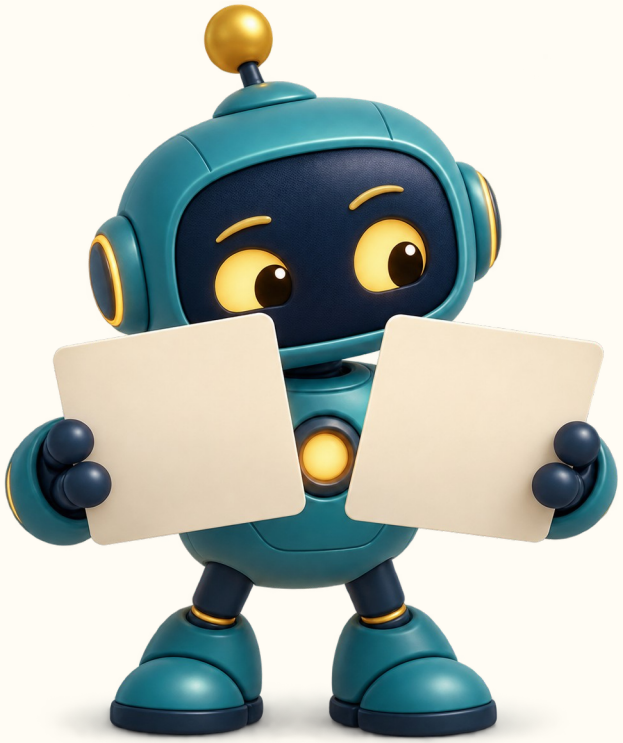
El resultado esperado y el real

Esperado

5, porque sumamos 2 y 3.

Real

5, el valor que devolvió.



Coinciden. Esta comprobación pasa.

Un resultado distinto

Esperado: 5

Real: 6

La diferencia hace fallar
la comprobación.



Los pasos de una prueba

1. Preparar

Elegir los datos: 2 y 3.

2. Ejecutar

Pedirle al programa que los sume.

3. Comprobar

Comparar el resultado con 5.

Qué hace PHPUnit

Vos escribís el caso

Decidís qué resultado debería producir el código.

PHPUnit lo ejecuta

Informa si las comprobaciones pasan o fallan.

La regla del sistema determina qué esperamos.

Un assert es una comprobación

```
$this->assertSame(5, $resultado);
```

5

Es el resultado esperado.

$resultado

Contiene el valor real.

Ejemplo 1: una suma

La calculadora de nuestro proyecto

app/Services/Calculadora.php

```
public function sumar(int $a, int $b): int
{
    return $a + $b;
}
```

Qué hace

Recibe dos números.

Devuelve su suma.

Este es el código que queremos comprobar.

Ejemplo 1: preparar, ejecutar y comprobar

La primera prueba de la suma

```
$calculadora = new Calculadora();
$resultado = $calculadora->sumar(2, 3);
$this->assertSame(5, $resultado);
```

La comparación

El método devolvió un valor.

El assert comprueba si ese valor es 5.

El resultado esperado se define antes de ejecutar.

Ejemplo 1: leer la comprobación

Cómo leer assertSame

```
$this->assertSame(
    5,
    $resultado
);
```

Esperado y real

Primer argumento: 5.

Segundo argumento: el resultado obtenido.

Deben coincidir en valor y tipo.

Ejemplo 1: tests/Unit/CalculadoraTest.php

El método que ejecuta PHPUnit

```
public function test_suma_positivos(): void
{
    $calculadora = new Calculadora();
    $resultado = $calculadora->sumar(2, 3);
    $this->assertSame(5, $resultado);
}
```

test_ identifica el caso que PHPUnit puede ejecutar.

Ejecutar el caso de la suma

Terminal del proyecto

```
php vendor/bin/phpunit --filter=test_suma_positivos --testdox
```

Suma positivos

OK: se cumplió la comprobación.

Ejemplo 1: reconocer un fallo

El assert detecta un cálculo incorrecto

Cambio temporal para la demostración

```
// Fallo provocado en la calculadora
return $a + $b + 1;
```

Esperado

5

Real

6

Corregimos el cálculo y volvemos a ejecutar.

Ejemplo 1: elegir otras entradas

Otros casos de la misma regla

Caso	Entrada	Esperado
Positivos	2 y 3	5
Número negativo	−2 y 3	1
Con cero	0 y 7	7

Cada caso tiene su propia entrada y expectativa.

Una prueba unitaria

Probamos el método sumar de una clase aislada.

Entrada: dos números.
Resultado: un número.

Ahora, las operaciones de un CRUD

Operación	Qué debería hacer
Crear	Guardar un cliente.
Leer	Mostrar los clientes.
Actualizar	Cambiar los datos del cliente elegido.
Eliminar	Quitar el cliente elegido.

Ejemplo 2: lo que debemos observar

La respuesta y los cambios en la base

Respuesta

La página o redirección que devolvió Laravel.

Base de datos

Los registros creados, modificados o eliminados.

La expectativa incluye lo que debe pasar con los datos.

Ejemplo 2: preparar la entrada

Los datos que elige la prueba

```
$datos = [
    'nombre' => 'Cliente de prueba',
    'email' => 'cliente@example.com'
];
```



Los campos vienen de tu CRUD. Los valores los elegís vos.

Una base de pruebas independiente

Cada caso prepara los registros que necesita.

RefreshDatabase mantiene independientes los casos.



Ejemplo 2: ejecutar una operación

La solicitud llega al controlador

Solicitud	Método	Operación
POST /clientes	store	Guardar
GET /clientes	index	Listar
PUT /clientes/{id}	update	Actualizar
DELETE /clientes/{id}	destroy	Eliminar

Ejemplo 2: tests/Feature/ClienteCrudTest.php

La clase de pruebas del CRUD

```
class ClienteCrudTest extends TestCase
{
    use RefreshDatabase;

    // Acá escribimos los casos.
}
```

Cada método test_ describe un caso independiente.

Ejemplo 2: guardar

El caso de guardar

Parte del caso	Qué definimos
Entrada	Nombre y email válidos en $datos.
Acción	Enviar POST /clientes.
Esperado	Los datos guardados y vuelta al listado.

Primero explicamos el caso. Después lo escribimos.

Preparar y enviar los datos

```
$datos = [
    'nombre' => 'Cliente de prueba',
    'email' => 'cliente@example.com'
];
$respuesta = $this->post(
    '/clientes', $datos
);
```

La entrada

Elegimos los valores en $datos.

Después enviamos esa misma variable.

Comprobar lo guardado

```
$this->assertDatabaseHas(
    'clientes', $datos
);
$this->assertDatabaseCount(
    'clientes', 1
);
```

Qué esperamos

Una fila con los mismos datos enviados.

Una cantidad total de 1.

La cantidad y el contenido responden a preguntas distintas.

Comprobar la respuesta

```
$respuesta->assertRedirect('/clientes');
```

Esperamos volver al listado de clientes.

$respuesta contiene lo que devolvió Laravel.

Cambiar los valores del ejemplo

```
$datos = [
    'nombre' => 'Otro ejemplo válido',
    'email' => 'otro.ejemplo@example.com'
];
```

La misma regla

Guardar el contenido que recibió el CRUD.

```
$this->assertDatabaseHas('clientes', $datos);
```

Ejecutar el caso de guardar

Extracto literal de una ejecución real de PHPUnit

```
$ php vendor/bin/phpunit --filter=test_guardar --testdox

Cliente Crud (Tests\Feature\ClienteCrud)
 ✔ Guardar
OK (1 test, 4 assertions)
```

El caso pasó porque se cumplieron sus condiciones.

Fallo: la base quedó vacía

Extracto literal de una ejecución real de PHPUnit

```
 ✘ Guardar
   | Failed asserting that a row in the table [clientes] matches the
attributes {
   |     "nombre": "Cliente de prueba",
   |     "email": "cliente@example.com"
   | }.
   | The table is empty.
FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

Esperábamos los datos enviados. No aparecieron en la base.

Listar: preparar un cliente

```
$datos = [
    'nombre' => 'Cliente de prueba',
    'email' => 'cliente@example.com'
];
Cliente::create($datos);
```

Antes de listar

Creamos el dato que después esperamos ver.

Cada caso prepara una situación independiente.

Listar: comprobar lo visible

```
$respuesta = $this->get('/clientes');
$respuesta->assertOk();
$respuesta->assertSee($datos['nombre']);
```

Qué revisamos

La página respondió con 200.

El nombre preparado aparece en su contenido.

Actualizar: elegir los datos nuevos

```
$cliente = Cliente::create($datos);
$nuevosDatos = [
    'nombre' => 'Cliente actualizado',
    'email' => 'actualizado@example.com'
];
```

Dos momentos

Un cliente ya creado.

Una entrada con los valores nuevos.

Actualizar: enviar al cliente elegido

```
$respuesta = $this->put(
    '/clientes/' . $cliente->id,
    $nuevosDatos
);
```

$cliente->id

Es el ID del registro que preparó la prueba.

La solicitud modifica ese registro.

El segundo argumento contiene los datos nuevos.

Actualizar: comprobar el mismo registro

```
$this->assertDatabaseHas('clientes', [
    'id' => $cliente->id,
    'nombre' => $nuevosDatos['nombre'],
    'email' => $nuevosDatos['email'],
]);
```

Qué esperamos

El mismo cliente.

Los valores nuevos que enviamos.

Actualizar conserva el registro y cambia sus datos.

Eliminar: ese ID debe desaparecer

```
$cliente = Cliente::create($datos);
$respuesta = $this->delete(
    '/clientes/' . $cliente->id
);
$this->assertDatabaseMissing(
    'clientes', ['id' => $cliente->id]
);
```

El registro del caso

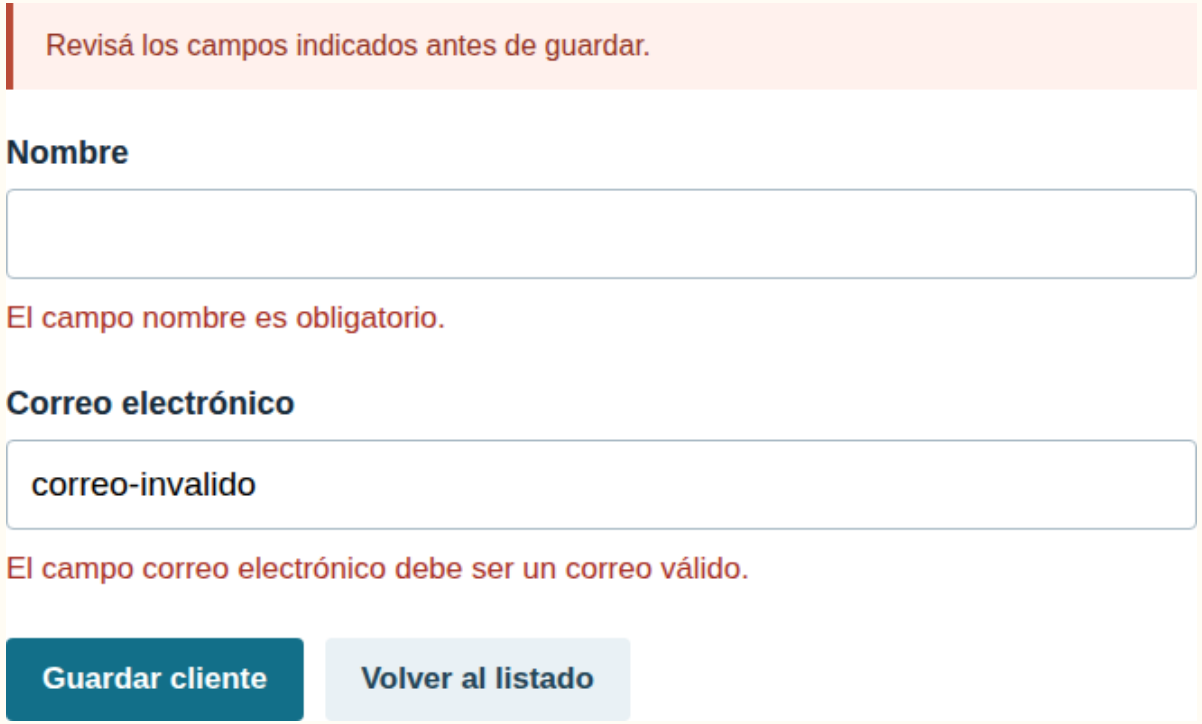
La prueba lo crea.

La operación lo elimina.

El assert busca su ausencia.

El ID sale del cliente creado por este caso.

Un caso con datos inválidos



Qué esperamos

Errores para nombre y email.

Ningún cliente guardado.

Comprobar el rechazo de la entrada

```
$respuesta->assertSessionHasErrors(
    ['nombre', 'email']
);
$this->assertDatabaseCount(
    'clientes', 0
);
```

Una respuesta correcta

Los campos tienen errores.

La base continúa sin clientes.

Los errores son el resultado esperado de este caso.

Los cinco casos básicos del CRUD

Extracto literal de una ejecución real de PHPUnit

```
$ php vendor/bin/phpunit tests/Feature/ClienteCrudTest.php --testdox

Cliente Crud (Tests\Feature\ClienteCrud)
 ✔ Guardar
 ✔ Listar
 ✔ Actualizar
 ✔ Eliminar
 ✔ Rechazar datos invalidos
OK (5 tests, 21 assertions)
```

El caso inválido pasa cuando el rechazo funciona.

El fundamento de los dos ejemplos

Unitarias y pruebas de integración

Ejemplo	Qué comprobamos	Tipo
Calculadora	Un método aislado.	Unitaria
CRUD de Cliente	Ruta, controlador y base.	Feature / integración

PHPUnit puede ejecutar ambos tipos de prueba.

Adaptar los casos a tu módulo

Parte	Qué adaptás
Rutas	Las URL de tu módulo.
Modelo y tabla	Los que utiliza tu sistema.
Campos y valores	Los del formulario, con entradas del caso.
Expectativa	El resultado que exige esa operación.

Actividades para practicar

1. Otra suma

Entradas 3 y 4. Definí primero el resultado esperado.

2. Guardar otros datos

Elegí valores propios y comprobá que se guardaron.

3. Una entrada inválida

Definí qué error esperás y qué debe pasar en la base.

Qué revisar cuando falla

Mensaje o situación	Qué revisar
Esperado 5, real 6	El cálculo y la expectativa.
Tabla vacía al guardar	Validación e instrucción de guardado.
Redirección al login	Usuario autorizado para el caso.
could not find driver	pdo_sqlite en el PHP de terminal.

Una expectativa clara

Definís qué debería pasar.
Ejecutás el código.
Comprobás lo que ocurrió.



Consulta: phpunit.xml

Configuración de la base de pruebas

```
<env name="APP_ENV" value="testing" force="true"/>
<env name="DB_CONNECTION" value="sqlite" force="true"/>
<env name="DB_DATABASE" value=":memory:" force="true"/>
<env name="DB_URL" value="" force="true"/>
```

Antes de ejecutar las pruebas

```
php artisan config:clear
```

La clase base de cada ejemplo

CalculadoraTest.php: prueba aislada

```
use PHPUnit\Framework\TestCase;
```

ClienteCrudTest.php: prueba con Laravel

```
use Tests\TestCase;
use Illuminate\Foundation\Testing\RefreshDatabase;
```

Documentación de apoyo

Laravel: inicio de las pruebas

`https://laravel.com/docs/12.x/testing`

Laravel: solicitudes y respuestas

`https://laravel.com/docs/12.x/http-tests`

Laravel: comprobar la base

`https://laravel.com/docs/12.x/database-testing`

PHPUnit: aserciones

`https://docs.phpunit.de/en/11.5/assertions.html`